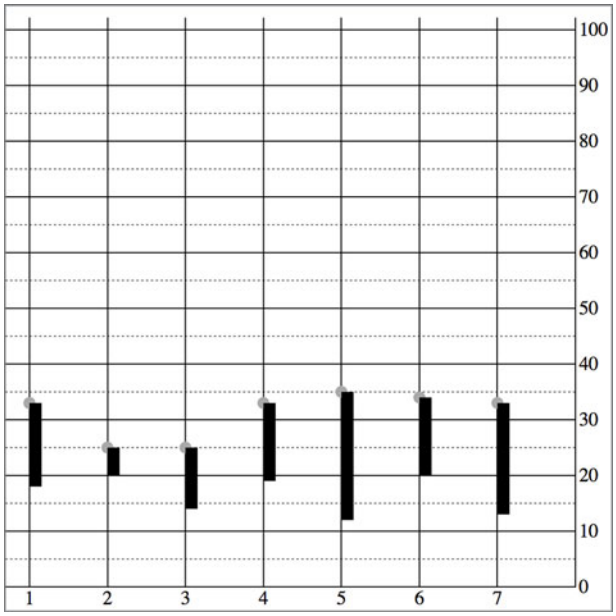


Figure 4.13 The `<rect>` elements represent the scaled range of the first and third quartiles of visitor age. They're placed on top of a gray `<circle>` in each `<g>` element, which is placed on the chart at the median age. The rectangles are drawn, as per SVG convention, from the `<g>` down and to the right.

and `<text>` elements. That's a clean method, and the only reasons to use `.each()` to add child elements are if you prefer the syntax, you plan on doing complex operations involving each data element, or you want to add conditional tests to change whether or what child elements you're appending. You can see how to use `.each()` to add child elements in action in the following listing, which takes advantage of the scales we created in listing 4.3 and draws rectangles on top of the circles we've already drawn.

Listing 4.4 Initial boxplot drawing code

```
d3.select("svg").selectAll("g.box")
    .data(data).enter()
    .append("g")
    .attr("class", "box")
    .attr("transform", function(d) {
        return "translate(" + xScale(d.day) +"," + yScale(d.median) + ")";
    }).each(function(d,i) {
            d3.select(this)
            .append("rect")
            .attr("width", 20)
            .attr("height", yScale(d.q1) - yScale(d.q3));
    });
```

The d and i variables are declared in the .each() anonymous function, so each time we access it, we get the data bound to the original element.

Because we're inside the .each(), we can select(this) to append new child elements.

The new rectangles indicating the distribution of visitor ages, as shown in figure 4.13, are not only offset to the right, but also showing the wrong values. Day 7, for instance, should range in value from 30 to 50, but instead is shown as ranging from 13 to 32. We know it's doing that because that's the way SVG draws rectangles. We have to update our code a bit to make it accurately reflect the distribution of visitor ages: